

euclid

```
// ax+by=gcd
int gcd(int a,int b,int&x,int&y){
    x=1;y=0;
    int x1=0,y1=1,a1=a,b1=b;
    while(b1){
        int q=a1/b1;
        tie(x,x1)=make_tuple(x1,x-q*x1);
        tie(y,y1)=make_tuple(y1,y-q*y1);
        tie(a1,b1)=make_tuple(b1,a1-q*b1);
    }
    return a1;
}
```

min_cost_flow

```
struct MinCostFlow{
    int n;
    vc<vi>adj,cst,cap;
    const int INF=1e9;

    MinCostFlow(int n):n(n){
        adj=vc(n,vi());
        cst=vc(n,vi(n,0));
        cap=vc(n,vi(n,0));
    }

    void add_edge(int from,int to,int capacity,int cost){
        adj[from].pb(to);
        adj[to].pb(from);
        cst[from][to]=cost;
        cst[to][from]=-cost;
        cap[from][to]=capacity;
    }

    void shortest_paths(int v0,vi&d,vi&p){
        d.assign(n,INF);
        d[v0]=0;
        vc<bool>inq(n,false);
        queue<int>q;
        q.push(v0);
        p.assign(n,-1);
        while(!q.empty()){
            int u=q.front();
            q.pop();
            inq[u]=false;
            for(int v:adj[u]){
                if (cap[u][v]>0&&d[v]>d[u]+cst[u][v]){
                    d[v]=d[u]+cst[u][v];
                    p[v]=u;
                    if(!inq[v]){
                        inq[v]=true;
                        q.push(v);
                    }
                }
            }
        }
    }

    int min_cost_flow(int K,int s,int t){
        int flow=0;
        int cost=0;
        vi d,p;
        while(flow<K){
            shortest_paths(s,d,p);
            if(d[t]==INF)
                break;
            int f=K-flow;
            int cur=t;
            while(cur!=s){
                f=min(f,cap[p[cur]][cur]);
                cur=p[cur];
            }
            flow+=f;
            cost+=f*d[t];
            cur=t;
            while(cur!=s){
                cap[p[cur]][cur]-=f;
                cap[cur][p[cur]]+=f;
                cur=p[cur];
            }
        }
        return flow<K?-1:cost;
    }
};
```

fft

```
using cd=complex<double>;
const double PI=acos(-1);

void fft(vc<cd>&a,bool inv){
    int n=(int)a.size();
    if(n==1)
        return;
    vc<cd>a0(n/2),a1(n/2);
    rep(i,(n+1)/2){
        a0[i]=a[2*i];
```

```

    a1[i]=a[2*i+1];
}
fft(a0,inv);
fft(a1,inv);
double ang=2*PI/n*(inv?-1:1);
cd w(1),wn(cos(ang),sin(ang));
rep(i,(n+1)/2){
    a[i]=a0[i]+w*a1[i];
    a[i+n/2]=a0[i]-w*a1[i];
    if(inv){
        a[i]/=2;
        a[i+n/2]/=2;
    }
    w*=wn;
}
}

// polynomial multiplication
vl multiply(vl&a,vl&b){
    vc<cd>fa(all(a)),fb(all(b));
    int n=1;
    while(n<sz(a)+sz(b))
        n<<=1;
    fa.resize(n);
    fb.resize(n);
    fft(fa,0);
    fft(fb,0);
    rep(i,n)
        fa[i]*=fb[i];
    fft(fa,1);
    vc<ll>res(n);
    rep(i,n)
        res[i]=round(fa[i].real());
    return res;
}

```

setup

```

/*
setxkbmap us -option caps:escape

~/.vimrc

set number
set relativenumber
syntax enable
set autoindent
set tabstop=2
set shiftwidth=2
set wrap
set scrolloff=20
filetype plugin indent on
autocmd FileType cpp setlocal cindent

Makefile

CXX = g++
CXXFLAGS = -std=c++20 -O2 -Wall -DLOC

%: %.cpp
    $(CXX) $(CXXFLAGS) $< -o $@

*/
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef long double ld;
#define fwd(i,a,b) for(int i=(a); i<(b);i++)
#define rep(i,n) fwd(i,0,n)
#define all(x) (x).begin(), (x).end()
#define sz(x)((int)(x).size())
#define f1 first
#define f2 second
#define pb push_back
#define vc vector
typedef vector<int> vi;
typedef vector<ll> vl;
typedef pair<int,int> pii;
typedef pair<ll,ll> pll;

#ifdef LOC
template<typename T1,typename T2>
auto& operator<<(ostream& out, pair<T1, T2> a) {return out<<("<a.st<<", "<a.nd<<");}
template<typename T>
auto& operator<<(ostream& out, vector<T>& a) {
    out<<"{";
    int cnt=0;
    for(auto b:a){
        cnt++;
        out<<b;
        if(cnt!=a.size())
            out<<",";
    }
    return out<<"}";
}

#define debug(x) cerr<<[" #x "]: "<x<<"\n";
#else
#define debug(x) ;
#endif

void solve(){
}

```

```

int main(){
    cin.tie(0);cout.tie(0);ios::sync_with_stdio(false);
    int t=1;cin>>t;
    while(t-->0)solve();
    return 0;
}

```

li_chao

```

struct LCT{
    int n;
    vc<pll>lines;

    LCT(int n):n(n){
        lines=vc<pll>(4*n,{0,1e18});
    }

    ll ev(pll a,int x){
        return a.f1*x+a.f2;
    }

    void add_line(pll nw,int v,int l,int r){
        int m=(l+r)/2;
        bool lef=ev(nw,l)<ev(lines[v],l);
        bool mid=ev(nw,m)<ev(lines[v],m);
        if(mid)
            swap(lines[v],nw);
        if(r-l==1)
            return;
        else if(lef!=mid)
            add_line(nw,2*v,l,m);
        else
            add_line(nw,2*v+1,m,r);
    }

    void add_line(pll nw){
        add_line(nw,1,0,n);
    }

    ll get(int x,int v,int l,int r){
        int m=(l+r)/2;
        if(r-l==1)
            return ev(lines[v],x);
        else if(x<m)
            return min(ev(lines[v],x),get(x,2*v,l,m));
        else
            return min(ev(lines[v],x),get(x,2*v+1,m,r));
    }

    ll get(int x){
        return get(x,1,0,n);
    }
};

```

hash_string

```

const int HMOD1=1e9+7;
const int HMOD2=1e9+9;
const int HX1=31;
const int HX2=29;

struct FastString{
    string s;
    vc<ll>h1,h2,p1,p2;
    FastString(string s):s(s){
        p1=p2=h1=h2={1};
        fwd(i,1,sz(s)+1){
            p1.pb((p1[i-1]*HX1)%HMOD1);
            p2.pb((p2[i-1]*HX2)%HMOD2);
            h1.pb((h1[i-1]+p1[i-1]*(s[i-1]-'a'+1))%HMOD1);
            h2.pb((h2[i-1]+p2[i-1]*(s[i-1]-'a'+1))%HMOD2);
        }
    }
    ll get(int l,int r){
        ll hsh1=(h1[r]+HMOD1-h1[l])%HMOD1*p1[sz(s)-l]%HMOD1;
        ll hsh2=(h2[r]+HMOD2-h2[l])%HMOD2*p2[sz(s)-l]%HMOD2;
        return hsh1+(hsh2<<32);
    }
};

```

tarjan_bridge

```

int timer,tin[maxN],low[maxN];
bool vis[maxN];
vi nodes[maxN];
vc<pii>bridges;

void tarjan_bridge(int u=0,int p=0){
    vis[u]=true;
    tin[u]=low[u]=++timer;
    for(int v:nodes[u]){
        if(v!=p){
            if(vis[v])
                low[u]=min(low[u],tin[v]);
            else{

```

```

        tarjan_bridge(v,u);
        low[u]=min(low[u],low[v]);
        if(low[v]>tin[u])
            bridges.push_back({u,v});
    }
}
}
}
}

```

max_flow

```

struct MaxFlow{
    int n;
    vc<vi>adj;
    vc<vl>cp;

    MaxFlow(int n):n(n){
        adj=vc<vc<int>>(n);
        cp=vc(n,vc(n,0LL));
    }

    // connect a and b with capacity c
    void add(int a,int b,int c){
        if(cp[a][b]==0){
            adj[a].pb(b);
            adj[b].pb(a);
        }
        cp[a][b]+=c;
    }

    ll bfs(int s,int t,vi&p){
        fill(all(p),-1);
        p[s]=-2;
        queue<pii>q;
        q.push({s,1e9});
        while(!q.empty()){
            int c=q.front().f1;
            ll f=q.front().f2;
            q.pop();
            for(int x:adj[c]){
                if(p[x]==-1&&cp[c][x]){
                    p[x]=c;
                    ll r=min(f,cp[c][x]);
                    if(x==t)
                        return r;
                    q.push({x,r});
                }
            }
        }
        return 0;
    }

    // call only once
    ll maxflow(int s,int t){
        ll f=0,r;
        vi pr(n);
        while((r=bfs(s,t,pr))){
            f+=r;
            int cu=t;
            while(cu!=s){
                int p=pr[cu];
                cp[p][cu]-=r;
                cp[cu][p]+=r;
                cu=p;
            }
        }
        return f;
    }
};

```

z_algorithm

```

// z[i] pove, da se z[i] znakov zacensi z i-tim ujemajo s prvimi z[i] znaki stringa s.
vi z_function(string s){
    vi z(sz(s));
    int l=0,r=0;
    fwd(1,1,sz(s)){
        if(i<r)
            z[i]=min(r-i,z[i-l]);
        while(i+z[i]<sz(s)&&s[z[i]]==s[i+z[i]])
            z[i]++;
        if(i+z[i]>r)
            l=i,r=i+z[i];
    }
    return z;
}

```

modulo_int

```

const int MOD=1e9+7,PC=1e7;
ll M_inv[PC+1];
struct M{
    ll val=0;
    static void precomp(){M_inv[1]=1;for(ll i=2;i<=PC;++i){
        M_inv[i]=(MOD-MOD/i)*M_inv[MOD%i]%MOD;if(M_inv[i]<0)M_inv[i]+=MOD;}}
}

```

```

static ll calc_inv(ll i){if(i<=PC)return M_inv[i];
return(MOD-MOD/i)*calc_inv(MOD%i)%MOD;}
M(ll x){val=(x%MOD+MOD)%MOD;}
M()=default;
template<class T>bool operator==(T b){return val==M(b).val;}
template<class T>bool operator!=(T b){return val!=M(b).val;}
template<class T>M operator+(T bt){M b(bt);M r(val);r.val+=b.val;if(r.val>=MOD)
r.val-=MOD;return r;}
template<class T>M operator-(T bt){M b(bt);M r(val);r.val+=MOD-b.val;if(r.val>=MOD)
r.val-=MOD;return r;}
template<class T>M operator*(T bt){M b(bt);M r(val);r.val*=b.val;r.val%=MOD;return r;}
template<class T>M&operator+=(T bt){M b(bt);*this=*this+b;return*this;}
template<class T>M&operator-=(T bt){M b(bt);*this=*this-b;return*this;}
template<class T>M&operator*=(T bt){M b(bt);*this=*this*b;return*this;}
M pow(ll e){M r(1);M a(val);while(e){if(e%2)r=r*a;a=a*a;e/=2;}return r;}
template<class T>M operator/(T bt){ M b(bt); return M(val*calc_inv(b.val)); }
template<class T>M&operator/=(T bt){M b(bt);*this*=calc_inv(b.val);return*this;}
};
namespace std{template<>struct hash<M>{
inline size_t operator()(const M&x)const{return x.val;}
};}
template<class T>bool operator<(T at,T bt){M b(bt);M a(at);return a.val<b.val;}
ostream&operator<<(ostream&s,M m){s<<m.val;return s;}
istream&operator>>(istream&s,M&m){s>>m.val;return s;}

```

fenwick

```

struct FT{
vl s;
FT(int n):s(n){}
void upd(int i,int x){
for(;i<sz(s);i|=i+1)s[i]+=x;
}
ll qry(int i){
ll r=0;
for(;i>0;i&=i-1)r+=s[i-1];
return r;
}
};

struct FT2{
vc<vi>ys;vc<FT>ft;
FT2(int h):ys(h){}
void inform(int x,int y){
for(;x<sz(ys);x|=x+1)ys[x].pb(y);
}
void init(){
for(vi&v:ys)sort(all(v)),ft.pb(sz(v));
}
int ind(int x,int y){
return(int)(lower_bound(all(ys[x]),y)-ys[x].begin());
}
void upd(int x,int y,int dif){
for(;x<sz(ys);x|=x+1)
ft[x].upd(ind(x,y),dif);
}
ll qry(int x,int y){
ll r=0;
for(;x;x&=x-1)
r+=ft[x-1].qry(ind(x-1,y));
return r;
}
ll qry(int x1,int y1,int x2,int y2){
return qry(x2,y2)-qry(x2,y1)-qry(x1,y2)+qry(x1,y1);
}
};

```

convex_hull

```

struct pt {
double x, y;
bool operator == (pt const& t) const {
return x == t.x && y == t.y;
}
};

int orientation(pt a, pt b, pt c) {
double v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
if (v < 0) return -1; // clockwise
if (v > 0) return +1; // counter-clockwise
return 0;
}

bool cw(pt a, pt b, pt c, bool include_collinear) {
int o = orientation(a, b, c);
return o < 0 || (include_collinear && o == 0);
}

bool collinear(pt a, pt b, pt c) { return orientation(a, b, c) == 0; }

void convex_hull(vector<pt>& a, bool include_collinear = false) {
pt p0 = *min_element(a.begin(), a.end(), [](pt a, pt b) {
return make_pair(a.y, a.x) < make_pair(b.y, b.x);
});
sort(a.begin(), a.end(), [&p0](const pt& a, const pt& b) {
int o = orientation(p0, a, b);
if (o == 0)
return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)*(p0.y-a.y)
< (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)*(p0.y-b.y);
return o < 0;
});
};

```

```

    if (include_collinear) {
        int i = (int)a.size()-1;
        while (i >= 0 && collinear(p0, a[i], a.back())) i--;
        reverse(a.begin()+i+1, a.end());
    }

    vector<pt> st;
    for (int i = 0; i < (int)a.size(); i++) {
        while (st.size() > 1 && !cw(st[st.size()-2], st.back(), a[i], include_collinear))
            st.pop_back();
        st.push_back(a[i]);
    }

    if (include_collinear == false && st.size() == 2 && st[0] == st[1])
        st.pop_back();

    a = st;
}

```

suffix_array

```

vi suffix_array(string s){
    s.pb('$');
    int n=sz(s);
    const int A=256;
    vi p(n),c(n),cnt(max(A,n),0);
    rep(i,n)
        cnt[s[i]]++;
    fwd(i,1,A)
        cnt[i]+=cnt[i-1];
    rep(i,n)
        p[--cnt[s[i]]]=i;
    c[p[0]]=0;
    int cl=1;
    fwd(i,1,n){
        if(s[p[i]]!=s[p[i-1]])
            cl++;
        c[p[i]]=cl-1;
    }
    vi pn(n),cn(n);
    for(int h=0;1<=h<n;h++){
        rep(i,n){
            pn[i]=p[i]-(1<=h);
            if(pn[i]<0)
                pn[i]+=n;
        }
        fill(cnt.begin(),cnt.begin()+cl,0);
        rep(i,n)
            cnt[c[pn[i]]]++;
        fwd(i,1,cl)
            cnt[i]+=cnt[i-1];
        for(int i=n-1;i>=0;i--){
            p[--cnt[c[pn[i]]]]=pn[i];
            cn[p[0]]=0;
            cl=1;
            fwd(i,1,n){
                pii cur={c[p[i]],c[(p[i]+(1<=h))%n]};
                pii prev={c[p[i-1]],c[(p[i-1]+(1<=h))%n]};
                if(cur!=prev)
                    cl++;
                cn[p[i]]=cl-1;
            }
            c.swap(cn);
        }
        p.erase(p.begin());
        return p;
    }
}

// p is suffix array (which is needed)
vi lcp(string s,vi p) {
    int n=sz(s);
    vi r(n,0);
    rep(i,n)
        r[p[i]]=i;
    int k=0;
    vi l(n-1,0);
    rep(i,n){
        if(r[i]==n-1){
            k=0;
            continue;
        }
        int j=p[r[i]+1];
        while(i+k<n&&j+k<n&&s[i+k]==s[j+k])
            k++;
        l[r[i]]=k;
        if(k)
            k--;
    }
    return l;
}

```